

TP 1

1 Python Installation

1.1 Using Jupyter online

Step 1 : <https://jupyter.unilim.fr>

Step 2 : Log in using Unilim Identification

Step 3 : click on 'New' then 'Python 3'

1.2 On your machine using Anaconda

Step 1 : Install Anaconda, <https://www.anaconda.com/products/distribution>

Step 2 : Open Qt console or Spyder

I give here some useful terminal commands to

- create a virtual environment from Terminal

Terminal

```
conda -V #check the version of conda installed on your machine
python -V #check the version of python installed on your machine
conda update conda #last update of conda
conda create -n envname python=x.x anaconda #create a virtual environment named
'envname' with the version python=x.x
conda activate envname #activate the virtual environment named envname
conda info -e #find the list of available environments
conda install -n yourenvname package #install additional package
conda deactivate #desactivate the virtual environment
conda remove -n envname -all #remove the virtual environment
```

- run a python script from Terminal

Terminal

```
pwd #find the the current directory
cd ~/Documents/Python/Kontio #change the directory
ls #show files in the directory
python #run python in a shell
python main.py #run the file main.py in the current directory
exit() #exit python
```

2 Pandas and Dataframe

```
1 import pandas as pd
2 import numpy as np
```

```
1 df_cat = pd.DataFrame(data = [[np.nan, np.nan, np.nan, 0], ['green', 'M', 'class1',
2                                     10],
3                               ['blue', 'L', 'class2', 20],
4                               ['white', 'M', 'class1', 30]],
5                               index=['A', 'B', 'C', 'D'])
df_cat.columns = ['color', 'size', 'classlabel', 'price']
```

	color	size	classlabel	price
A	NaN	NaN	NaN	0
B	green	M	class1	10
C	blue	L	class2	20
D	white	M	class1	30

```
1 df_cat.info()
2 print(df_cat.head(2))
3
4 print(df_cat.iloc[0])
5 print(df_cat.iloc[1])
6 print(df_cat.loc['A'])
7 print(df_cat.loc['B'])
8
9 print(df_cat['color'])
10
11 dir(df_cat)
```

3 From raw data and pre-processing to data set

In order to be a data scientist you need data. In fact, as a data scientist you will spend an embarrassingly large fraction of your time acquiring and cleaning (for more details, read *Data science for scratch, chap 9-10* written by Joel Grus).

Since learning algorithms are only applied to (x, y) vectors, we need to transform $(u, v) \in \mathcal{U} \times \mathcal{V}$ into vectors using embeddings. Typically a data u is a tuple, or list, $u = (u_1, \dots, u_n)$. Each component (or field) of u has a type, e.g., real number, Boolean, categorical, ordinal, word, text, audio, image, parse tree.

We consider with feature maps $\phi : \mathcal{U} \times \mathcal{V} \rightarrow \mathbb{R}^d$ that transform data into vectors (used as input/output data for the machine learning algorithm). Feature maps usually work on each field separately,

$$\phi(u_1, \dots, u_r, v_1, \dots, v_l) = (\phi_1(u_1), \dots, \phi_r(u_r), \phi_{r+1}(v_1), \dots, \phi_{r+l}(v_l)).$$

ϕ_i is an embedding of the type of field i into a vector. The choice of ϕ can directly affects the ability of our model to learn; therefore, it is extremely important that we preprocess our data before feeding it into our model.

In the example of Section 2, $\mathcal{U} = \{\text{all the colors}\} \times \{\text{all the sizes}\} \times \{\text{all the class labels}\}$ and $\mathcal{V} = \mathbb{R}$. Moreover, $r = 3$ and $l = 1$. We give here some simple embeddings and pre-processing techniques :

- For a number input $u \in \mathcal{U} = \mathbb{R}$, $\phi(u) = u$

```
1 df_cat['price']
```

```
0    10
1    20
2    30
Name: price, dtype: int64
```

- Handling Null Values : in any real-world dataset, there are always few null values. It doesn't really matter whether it is a regression, classification or any other kind of problem, no model can handle these NULL or NaN values on its own so we need to intervene.

```
1 df=df_cat.fillna(0)
2 print(df)
3 print(df_cat)
4
5 df_cat.fillna(0,inplace=True)
6 print(df_cat)
```

```
Name: color, dtype: object
```

	color	size	classlabel	price
A	0	0	0	0
B	green	M	class1	10
C	blue	L	class2	20
D	white	M	class1	30

	color	size	classlabel	price
A	NaN	NaN	NaN	0
B	green	M	class1	10
C	blue	L	class2	20
D	white	M	class1	30

	color	size	classlabel	price
A	0	0	0	0
B	green	M	class1	10
C	blue	L	class2	20
D	white	M	class1	30

```
1 df_cat.drop(['A'], inplace=True)
```

	color	size	classlabel	price
B	green	M	class1	10
C	blue	L	class2	20
D	white	M	class1	30

- Handling Categorical Variables. Categorical variables are basically the variables that are discrete and not continuous. Ex: color of an item is a discrete variable whereas its price is a continuous variable.

- Ordinal categorical variables : these variables can be ordered. Example : Size of a T-shirt. We can say that $M < L < XL$

```
1 !pip install scikit-learn # if you use Jupyter hub online
2 from sklearn.preprocessing import LabelEncoder #using Label Encoder class_le =
   LabelEncoder()
```

```
1 size_mapping = {'M':1, 'L':2} # using map() function
2 type(size_mapping) # Dictionary
3 size_mapping['M']
4 size_mapping['T']
5 T=size_mapping.get('T',0) #Dictionaries have a get method that returns a
   default value when you look up a key that is not in the dictionary
6 len(size_mapping)
7 size_mapping['S']=0
8 len(size_mapping)
9
10 df_cat['size'] = df_cat['size'].map(size_mapping)
11
12 class_le = LabelEncoder() #using Label Encoder
13 df_cat['classlabel'] = class_le.fit_transform(df_cat['classlabel'].values)
```

	color	size	classlabel	price
0	green	1.0	0	10
1	blue	2.0	1	20
2	white	1.0	0	30

- Nominal categorical variables : if you use the same map() function or LabelEncoder with nominal variables then the model will think that there is some sort of relationship between the nominal CVs. For instance, if we use map() to map the colors like

```
1 col_mapping = {'blue':1, 'green':2}
```

then according to the model, Green > Blue, which is a senseless assumption and the model will give you results considering this relationship. The correct way of handling nominal CVs is to use One-Hot Encoding. The easiest way to use One-Hot Encoding is to use the *get_dummies()* function

```
1 df_cat = pd.get_dummies(df_cat[['color','size','classlabel','price']])
```

	size	classlabel	price	color_blue	color_green	color_white
0	1.0	0	10	0	1	0
1	2.0	1	20	1	0	0
2	1.0	0	30	0	0	1

If we want to take into account that blue , green and white are colors, we could use their (R, G, B) ¹ description. For instance, blue : (0,0,255), green : (0,255,0) and white : (255,255,255)

	R	G	B	size	classlabel	price
0	0	0	255	1.0	0	10
1	0	255	0	2.0	1	20
2	255	255	255	1.0	0	30

- Other embeddings are more sophisticated : pre-trained neural network for images (maps images into vectors) and word2vec (maps words into vectors).
- Standardization : in order for our machine learning to work well as linear regression, it is very necessary for the data to have the same scale (=the same weight) in terms of features. Thus, we transform our values such that the mean of the values is 0 and the standard deviation is 1.

```
1 from sklearn.preprocessing import StandardScaler
```

```
1 std = StandardScaler()
2 for index, mot in enumerate(df_cat.columns) :
3     X = std.fit_transform(df_cat[mot].values.reshape(-1,1))
4     del df_cat[mot]
5     df_cat.insert(index, mot, X)
```

	R	G	B	size	classlabel	price
0	-0.707107	-1.414214	0.707107	-0.707107	-0.707107	-1.224745
1	-0.707107	0.707107	-1.414214	1.414214	1.414214	0.000000
2	1.414214	0.707107	0.707107	-0.707107	-0.707107	1.224745

4 Visualization

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import numpy as np
4 import seaborn as sns
5
6 fig, axes=plt.subplots(3,3)
7 fig.set_size_inches(15,15)
8 style=dict(color='C1',linewidth=1,linestyle='—',marker='.',mec='C2')
9
10 #DATA
11 data1,data2=np.random.randn(2,5000)
12 data3=pd.DataFrame({'Column1': [1,2,3,4], 'Column2':[9,8,7,6]})
13
14 #HISTOGRAMME
15 axes[0,0].hist(data1,color='C7',bins=50)
16
17 #LINEPLOT
```

¹Red, Green, Blue

```

18 axes[0,1].plot(data3.Column1,data3.Column2, **style)
19
20
21 #SCATTER PLOT
22 axes[0,2].scatter(data3.Column1,data3.Column2, color='C7')
23
24 #KDE : kernel density estimation (densite de probabilite)
25 sns.kdeplot(x=[1,23,58,94,67,81,46,2,54,76],y=[1,2,3,4,5,6,7,8,9,10], fill=True,
26           thresh=0, ax=axes[1,0])
27
28 #BAR PLOT
29 axes[1,1].bar(['Dog','Cat','Panda','Monkey'],[5,10,2,6],color='C4')
30
31
32 #PIE CHART
33 axes[1,2].pie([5,10,2,6], labels=['Dog','Cat','Panda','Monkey'], autopct='%1.0f
34           %%', explode=(0.0,0.15,0.0,0.0))
35 axes[1,2].set_title('Animal distributions')
36
37 #BOX PLOT (boite a moustache)
38 axes[2,0].boxplot(data2,showmeans=True,meanline=True)
39
40 axes[2,1].set_visible(False)
41 axes[2,2].set_visible(False)
plt.show()

```

5 Splitting data set : train, validation, test

In most cases, the data set is splitted into three parts :

- the training set, on which learning models will be estimated
- the validation set, to estimate hyper parameters (all learning techniques have some)
- the testing set, to evaluate the performance of the final model (formally, the test set can only be used once!) An illustration is given in Figure 1 and Figure 2

Cross-validation is often preferred to use a maximal amount of training data, and reduce the variability of the validation procedure: the available data are divided in k folds (typically $k = 5$ or 10), and all models are estimated k times, each time choosing a different fold as validation data (pink data below), and averaging the k obtained error measures. Cross-validation can be applied to any learning methods, and its detailed theoretical analysis is an active area of research (see, Arlot and Celisse, 2010, and the many references therein).

```

1 import numpy as np
2 from sklearn.model_selection import train_test_split

```

```

1 X, y = np.arange(10).reshape((5, 2)), range(5)
2 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
3           random_state=42)
4 print('X=',X)
5 print('y=',list(y))
6 print('X_train=',X_train)
7 print('X_test=',X_test)

```

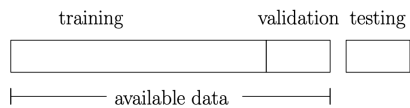
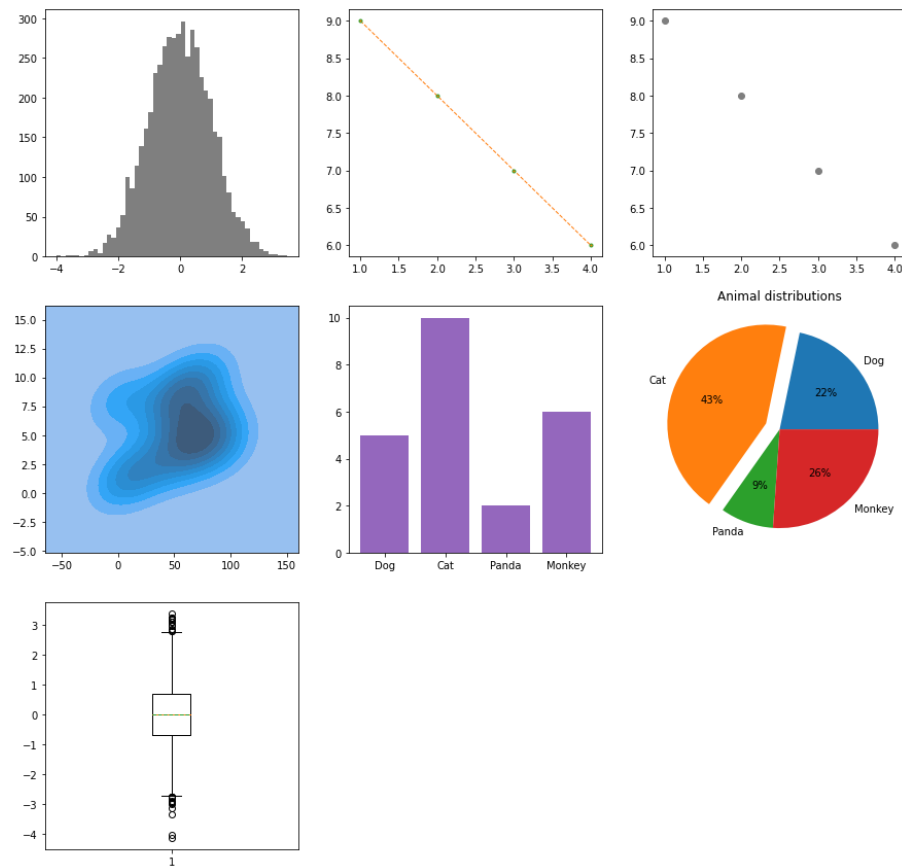


Figure 1: Splitting of input/output data

```

7 print('y_train=', y_train)
8 print('y_test=', y_test)

```

```

X= [[0 1]
     [2 3]
     [4 5]
     [6 7]
     [8 9]]
y= [0, 1, 2, 3, 4]
X_train= [[4 5]
           [0 1]
           [6 7]]

```

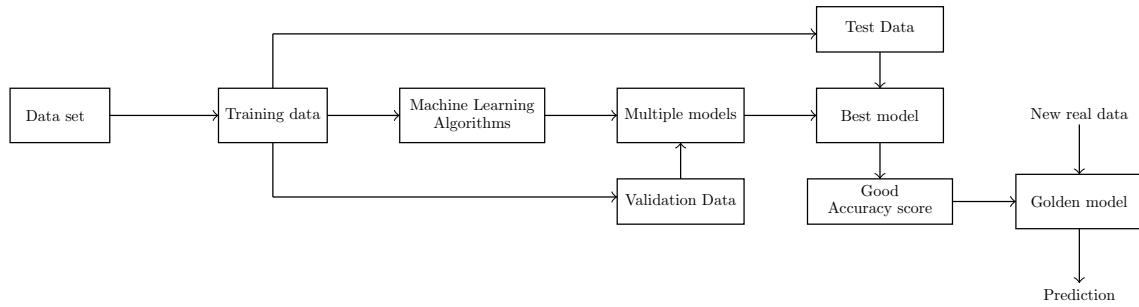


Figure 2: Machine learning pipeline with splitting data set



Figure 3: Splitting of input/output data with cross-validation

```

X_test= [[2 3]
 [8 9]]
y_train= [2, 0, 3]
y_test= [1, 4]
  
```